UNIVERSITY OF COLORADO - BOULDER
Robotics Program
ROBO 5302 (CSCI 4302/5302) - Advanced Robotics
Homework #4 (Assigned: Tue 11/4, Due: Fri 11/21 11:59pm on Canvas)
State Estimation

Isaac Edelman
48hr extension applied

Bayes Filter

Approach

For the Bayes Filter code, I implemented the 'predict' and 'update' functions inside bayes_filter.py. In my implementation, I used the Bayes filter to keep track of the robot's belief about where it is in the GridWorld. The filter maintains a probability distribution over every free cell in the map. In my code, this distribution is stored in self.belief, which represents the Bayes filter belief bel(x_t).

The Bayes filter proceeds in two steps: prediction and update, and these steps map directly onto my predict() and update() functions.

Prediction Step

In the prediction step, I apply the robot's motion model to compute the predicted belief, written as bel_bar(x_t). This represents what the robot's belief should be after executing an action but before looking at the new sensor readings. In my code, the predicted belief corresponds exactly to the temporary array: new_belief

This array starts filled with zeros and is updated according to the transition model. The motion model accounts for uncertainty: when the robot tries to move forward, it may move successfully with probability (1 - motion_noise), or the action may fail with probability (motion_noise). I also assume the robot's orientation is unknown, so I consider all four possible headings, each weighted by 0.25.

Mathematically, the prediction step of the Bayes filter is:

$$\text{bel\_bar}(x_t) = \text{sum}_{x_{t-1}}\, p(x_t \mid u_t, x_{t-1}) * \text{bel}(x_{t-1})$$

After computing new_belief, I normalize it and store it back into:

self.belief = new_belief \ np.sum(new_belief)

which completes the prediction phase.

Update Step

The update step incorporates the new sensor readings into the belief. This produces the corrected belief, denoted bel(x_t). In code, this is the updated value of self.belief after multiplying by the measurement likelihoods. In the update function, I compute a likelihood value for each cell by comparing the expected readings, computed with _get_expected_readings(), and the actual sensor measurements, passed into the function.

For every cell, I consider all four possible headings, compute how well the expected readings match the actual ones, and average the likelihood over headings. The measurement model rewards small differences (within a tolerance) and penalizes large differences.

The Bayes filter update rule is:

$$bel(x_t) = \eta * p(z_t \mid x_t) * bel\_bar(x_t)$$

In code, this corresponds to:

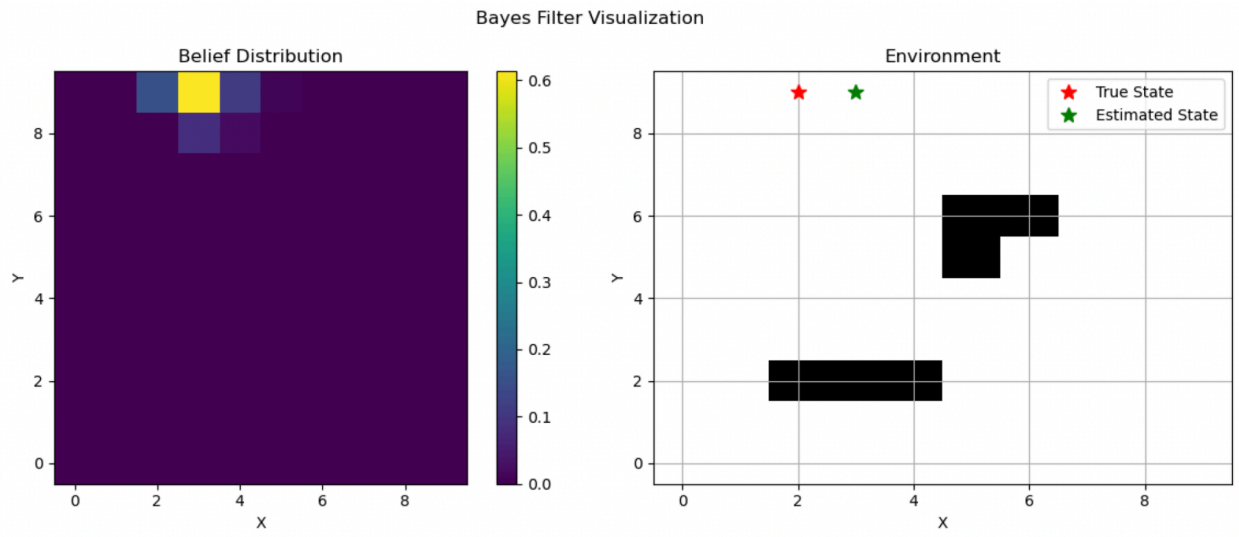
```
self.belief *= likelihood
self.belief /= np.sum(self.belief)
```

This normalization ensures all probabilities sum to 1.

Issues

Most of the challenges I ran into were related to the implementation logic rather than the underlying Bayes filter mathematics. One difficulty was determining how to correctly check whether a predicted move was valid during the prediction step. For this, I had to look into how self.env worked and understand how to properly reference and interpret the environment's grid structure. Another challenge was figuring out how to update the belief state correctly for both valid and invalid moves to make sure that the probability was updating the appropriate cells according to the motion model and noise assumptions.

Final Bayes Filter Visualization



EKF Filter

Approach

For the EFK implementation I estimated the robot's position and orientation in the 2D environment using range measurements. The robot follows a nonlinear unicycle motion model, with forward and angular velocities as control inputs. The environment includes a terrain-dependent velocity scaling based on:

terrain = np.sin(self.terrain_frequency * x) * np.cos(self.terrain_frequency * y)

To simulate terrain effects.

This made the motion model nonlinear and position-dependent. My implementation had to integrate both the nonlinear motion dynamics and noisy sensor measurements. To do this I implemented the 'motion_model()', 'motion_jacobian()', 'measurment_jacobian()', 'predict()', and 'update()' functions.

Motion Model

I first implemented the _motion_model() function, which predicts the next state given the current state, control inputs, and time step. In this function, I accounted for terrain effects by scaling the forward velocity according to the robot's position using the environment's velocity scaling function. I also added a speed-dependent turning factor that adjusts the angular velocity based on the forward speed, preventing instability at high speeds. This function outputs the predicted next state, which serves as the input for both the state update and the computation of the Jacobian matrix.

Motion Jacobian

Next, I wrote the _motion_jacobian() function, which linearizes the nonlinear motion model around the current state. This Jacobian matrix captures how small changes in x, y, and θ influence the predicted state. Since the velocity scaling depends on position, I numerically approximated its partial derivatives with respect to x and y. Using these derivatives I filled in the 3×3 Jacobian matrix according to the unicycle model. This Jacobian is then used in the EKF prediction step to update the state covariance.

Measurement Jacobian

I then worked on the _measurement_jacobian(). In _measurement_jacobian(), I derived the partial derivatives of the range measurement with respect to the robot's state, which linearizes the measurement model around the current estimate. I included a check for near-zero distances to avoid division by zero. These functions together allow the EKF to relate the state uncertainty to the measurement uncertainty during the update step.

Predict Step

The predict() function performs the EKF prediction step. I used the nonlinear motion model to update the robot's state and compute the Jacobian to propagate the covariance forward in time. After updating the state and covariance, I applied constraints on the covariance diagonal to prevent numerical instabilities and normalized the robot's heading to remain within $[0,2\pi)$.

Update Step

The update() function implements the EKF correction step using the range measurement. In this step, I first compute the expected measurement and its Jacobian. Then I calculate the Kalman gain, which determines how much the measurement should influence the state estimate. I apply the state correction while limiting the magnitude of the update to prevent destabilizing jumps. Finally, I update the covariance using the Joseph form to maintain numerical stability and append the updated state and covariance to the history.

Issues

One issue I had was correctly computing the Jacobian. I had to reference some Jacobian code to make sure that it was correct. I also had to look up how to properly implement the Joseph form of the covariance in python.

Derivation for the Jacobians for the F and H matrices

Since I am using the Unicycle model we can start with our state vector as:

$$X = [x\ y\ \theta]^T$$

And the control input vector as:

$$U = [v\ w]^T$$

Next we can look at the motion update model:

$$x_{t+1} = x_t + v\cos(\theta_t)\Delta t$$
$$y_{t+1} = y_t + v\cos(\theta_t)\Delta t$$
$$\theta_{t+1} = \theta_t + w\Delta t$$

For the Jacobian of the motion model (F) we want to take f with respect to the state x so:

$$F = \frac{\partial f}{\partial x} =$$

$\frac{\partial x_{t+1}}{\partial x}$	$\frac{\partial x_{t+1}}{\partial y}$	$\frac{\partial x_{t+1}}{\partial \theta}$
$\frac{\partial y_{t+1}}{\partial x}$	$\frac{\partial y_{t+1}}{\partial y}$	$\frac{\partial y_{t+1}}{\partial \theta}$
$\frac{\partial \theta_{t+1}}{\partial x}$	$\frac{\partial \theta_{t+1}}{\partial y}$	$\frac{\partial \theta_{t+1}}{\partial \theta}$

From this we can find the Jacobian F as:

$$F =$$

1	0	$-v\sin(\theta)\Delta t$
0	1	$v\cos(\theta)\Delta t$
0	0	1

For the Jacobian of the Measurement Model H we can first assume the predicted range for the sensor to be:

$$r = \sqrt{(x - x_s)^2 + (y - y_s)^2}$$

The Jacobian H is:

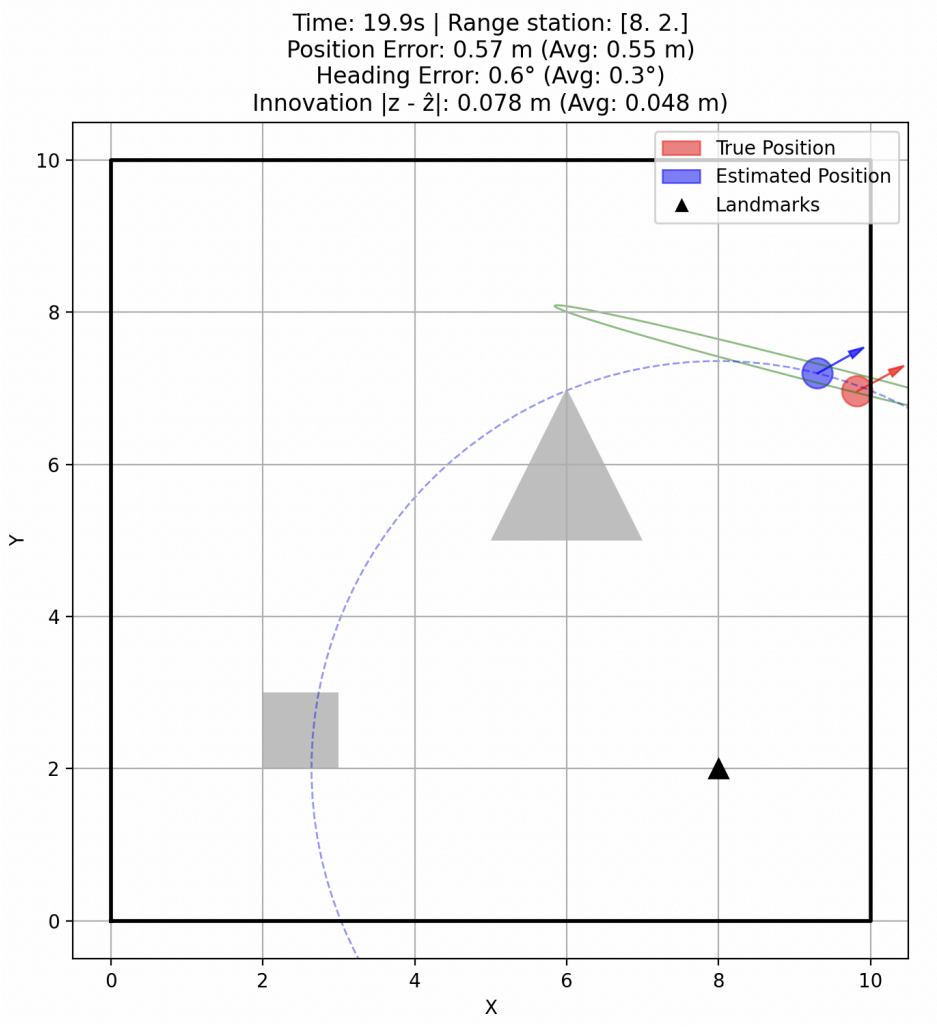
$$H = \frac{\partial h}{\partial x} = [\frac{\partial z}{\partial x} \ \frac{\partial z}{\partial y} \ \frac{\partial z}{\partial \theta}]$$

Applying the partial derivatives we get the following H matrix:

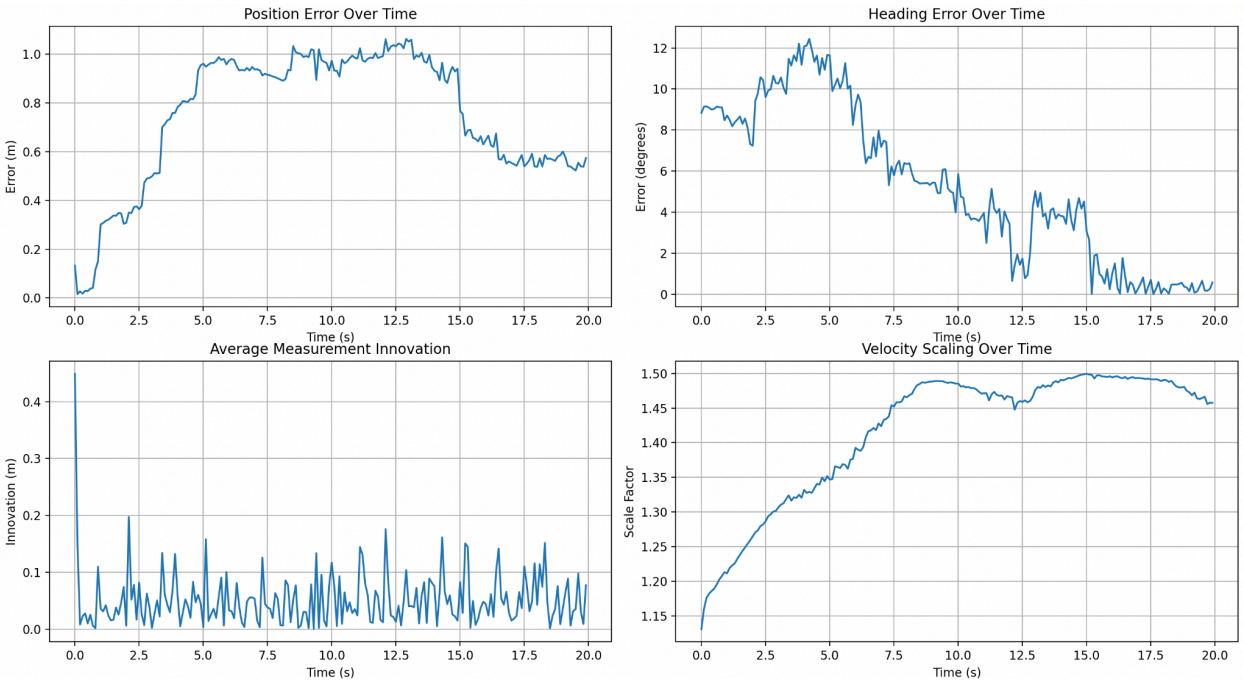
$$H = [\frac{x_s - x}{z} \ \frac{y_s - y}{z} \ 0]$$

If we have multiple beams from the sensor then H will have a row per beam.

Final EKF Vizulation



EKF Final Position Error over time, Heading Error over time, Average Measurement Innovation over time, and Velocity Scaling over time plots.



Particle Filter

Approach

For the particle filter I implemented the 'predict()', 'update()', 'resample()', and estimate_state()' functions. For the particle filter we want to localize a robot in a 2D environment. The robot state is represented by $[x,y,\theta]^T$, where x and y denote the position, and θ represents the heading angle. The control inputs are the forward and angular velocities. Since the environment can generate highly nonlinear and multimodal beliefs, a Particle Filter is more suitable than a Kalman Filter because it does not assume Gaussian distributions and can maintain multiple hypotheses simultaneously.

Predict

I started with the predict() function which implements the motion update for each particle. I propagated each particle through the robot's unicycle motion model while adding Gaussian noise to the control inputs. The noise variances α1 and α2 scale with the magnitude of the forward and angular velocities, reflecting that faster movements introduce more uncertainty. After updating the particle positions and headings, I checked for collisions. If a particle ended up in an invalid location, I resampled it randomly within free space. I also normalized headings to remain within

$[-\pi,\pi]$. This function allowed the particles to evolve over time according to the robot's motion while keeping an uncertainty.

Update Step

For the update() function we incorporate the sensor measurements into the particle belief. For each particle, I computed expected sensor measurements using the environment's range sensor model and compared them to the actual measurements. I calculated the likelihood of each particle based on the difference between expected and observed measurements, assuming Gaussian noise in the sensor readings. Each particle's weight was multiplied by its likelihood, and then the weights were normalized. This step made sure that the particles that are consistent with measurements receive higher weights allowing the function to update the belief distribution to match the observed environment.

Resample

Next I coded the resample() function, I implemented multinomial resampling to focus on the most likely particles. Particles were sampled according to their weights, and small Gaussian noise was added to maintain diversity and avoid particle 'depletion'. After resampling, all particle weights were reset to a uniform distribution. This step preserves the overall belief while concentrating particles in regions of higher probability.

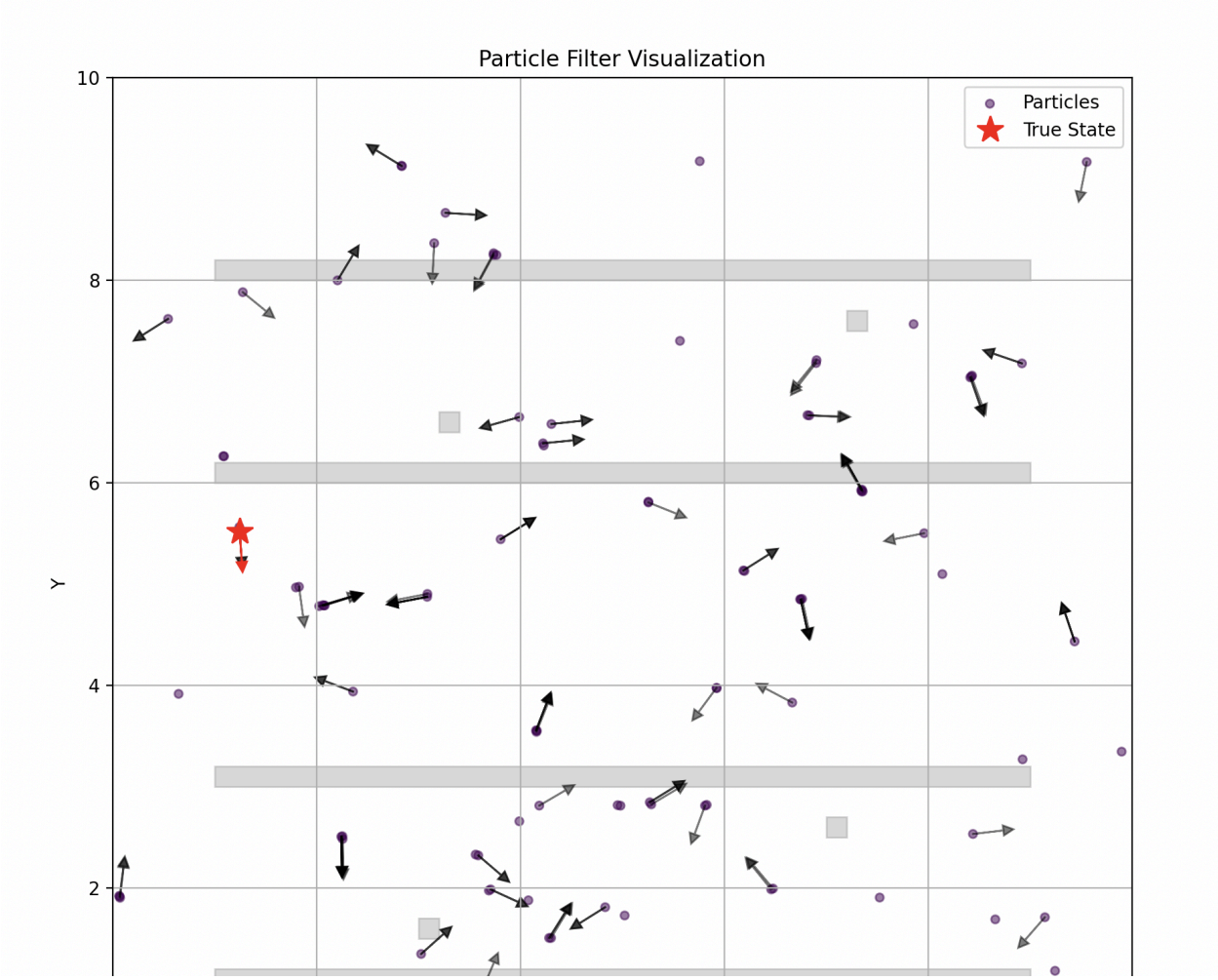
Estimate State

Finally, the estimate_state() function computes a point estimate from the particle set. For the x and y coordinates, I used a weighted average of particle positions. For the heading, I converted the angles to unit vectors, averaging them, and converting back to an angle. This weighted mean provided a summary of the robot's belief at each timestep.

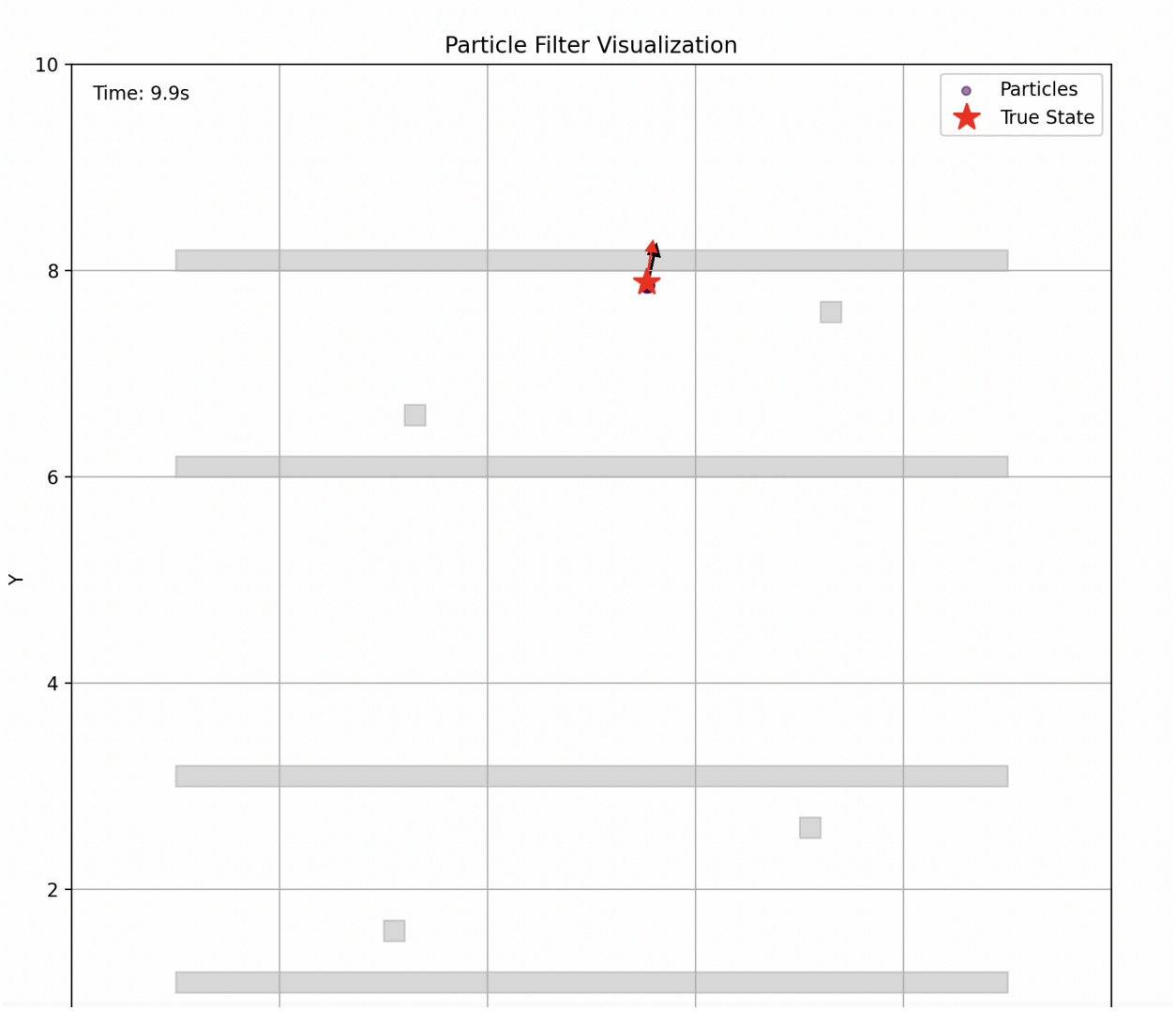
Issues

For the particle filter I had to look into what version of the gaussian distribution to use since we normalized it later. I ended up just using the e^ term since they are mathematically equivalent if eventually normalized which it is. I think I liked the particle filter implementation the most just due to its ease of implementation and method.

An initial step of the particle filter for reference



Final Particle filter visualization


Particle Filter Visualization
Time: 9.9s
Particles
True State
Y
10
8
6
4
2